
CUDA 실습

윤승현

동국대학교 컴퓨터·AI학부

행렬의 곱셈

- 32×32 **이상**의 두 행렬에 대하여,
 - CPU를 이용하여 두 행렬의 곱을 구하는 프로그램을 작성하고 시간을 측정
 - GPU를 이용하여 두 행렬의 곱을 구하는 프로그램을 작성하고 시간을 측정
 - 실행 시간 비교하여 GPU의 성능이 CPU의 성능보다 빨라지는 행렬의 크기를 조사
 - 단, 실행 시간은 release 모드에서 측정

행렬의 곱셈(CPU)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void MultMatCPU(float *P, float *M, float *N, int width);

void main()
{
    int width = 32;
    float *M = new float[width * width];
    float *N = new float[width * width];
    float *P = new float[width * width];
    for (int i = 0; i < width * width; ++i)
    {
        M[i] = rand() % 3 - 1;
        N[i] = rand() % 3 - 1;
        P[i] = 0.0;
    }

    clock_t st = clock();
    MultMatCPU(P, M, N, width);
    clock_t ed = clock();
    printf("Elapsed time: %u ns\n", ed - st);

    delete[] M;
    delete[] N;
    delete[] P;
}
```

```
void MultMatCPU(float *P, float *M, float *N, int width)
{
    for (int i = 0; i < width; ++i)
    {
        for (int j = 0; j < width; ++j)
        {
            float sum = 0.0;
            for (int k = 0; k < width; ++k)
            {
                float a = M[i * width + k];
                float b = N[k * width + j];
                sum += a * b;
            }
            P[i * width + j] = sum;
        }
    }
}
```

행렬의 곱셈(GPU)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "cuda_runtime.h"
#include "device_launch_parameters.h"

#define TILE_WIDTH 32

__global__ void MultMatGPU(float *P, float *M, float *N, int width);

void main()
{
    int width = 1024;
    float *P = new float[width * width];
    float *M = new float[width * width];
    float *N = new float[width * width];
    for (int i = 0; i < width * width; ++i)
    {
        M[i] = rand() % 3 - 1.0;
        N[i] = rand() % 3 - 1.0;
        P[i] = 0.0;
    }

    cudaError_t cudaStatus = cudaSetDevice(0);
    float *dev_P, *dev_M, *dev_N;
    cudaStatus = cudaMalloc((void **)&dev_P, width * width * sizeof(float));
    cudaStatus = cudaMalloc((void **)&dev_M, width * width * sizeof(float));
    cudaStatus = cudaMalloc((void **)&dev_N, width * width * sizeof(float));
    cudaStatus = cudaMemcpy(dev_M, M, width * width * sizeof(float), cudaMemcpyHostToDevice);
    cudaStatus = cudaMemcpy(dev_N, N, width * width * sizeof(float), cudaMemcpyHostToDevice);
```

행렬의 곱셈(GPU)

```
dim3 dimGrid((width - 1) / TILE_WIDTH + 1, (width - 1) / TILE_WIDTH + 1);
dim3 dimBlock(TILE_WIDTH, TILE_WIDTH);
clock_t st = clock();
MultMatGPU<<<dimGrid, dimBlock>>>(dev_P, dev_M, dev_N, width);
cudaDeviceSynchronize();
clock_t ed = clock();
printf("Elapsed time = %u ms\n", ed - st);

cudaStatus = cudaMemcpy(P, dev_P, width * width * sizeof(float), cudaMemcpyDeviceToHost);
cudaStatus = cudaDeviceReset();

delete[] P;
delete[] M;
delete[] N;
cudaFree(dev_P);
cudaFree(dev_M);
cudaFree(dev_N);
}
```

행렬의 곱셈(GPU)

```
__global__ void MultMatGPU(float *P, float *M, float *N, int width)
{
    int i = blockIdx.y * TILE_WIDTH + threadIdx.y;
    int j = blockIdx.x * TILE_WIDTH + threadIdx.x;

    if (i < width && j < width)
    {
        float sum = 0.0;
        for (int k = 0; k < width; ++k)
            sum += M[i * width + k] * N[k * width + j];

        P[i * width + j] = sum;
    }
}
```

Q&A